

Soluții — Interfețe C#

stefan-charp-interfaces-starter · runda 2 · 2026-07-28 · însoțește CODE_REVIEW.md — codul tău rămâne neatins, aici sunt variantele corectate, pe numerele din review

● B1 — Robot: reîncărcarea aparține lui PoateLivra

ex3/Models/Robot.cs

Contractul în doi timpi (`PoateLivra` → `Livreaza`) promite: dacă primul răspunde `true` , al doilea reușește. Reîncărcarea e un motiv de refuz, deci stă în `PoateLivra` — centrul sare la următorul livrator, nu crapă.

```
public class Robot : ILivrator
{
    public string Marca { get; }
    public int Capacitate { get; }

    private int livrare = 0;

    public Robot(string marca)
    {
        Marca = marca;
        Capacitate = 50;
    }

    public string Identificare()
    {
        return "Robot " + Marca;
    }

    public bool PoateLivra(double greutateKg)
    {
        if (livrare == 3)
        {
            livrare = 0;
            return false;
        }

        return greutateKg <= Capacitate;
    }

    public void Livreaza(string adresa, double greutateKg)
    {
        if (greutateKg > Capacitate)
        {
            throw new InvalidOperationException("This courier cannot deliver the package");
        }

        livrare++;
        Console.WriteLine(Identificare() + " livreaza la adresa " + adresa + " un colet de "
    }
}
```

Subtilitate: În `Livreaza` nu mai chem `PoateLivra`, ci verific direct greutatea. De ce? `PoateLivra` are acum efect secundar (consumă refuzul și resetează contorul) — dacă `Livreaza` l-ar chema și el, un singur colet ar consuma două întrebări și contorul ar minți. O metodă întrebare (*query*) care mută starea e mereu un semnal de atenție; aici e cerută de enunț, deci o izolăm într-un singur loc.

● B2 — NotificatorCulstoric: Decorator adevărat

`ex4/Models/NotificatorCuIstoric.cs`

Decoratorul semnează *același* contract ca obiectul pe care îl îmbracă, primește UN singur `INotificator` interior, delegă apelul și adaugă comportamentul propriu (memorarea mesajului). Din exteriorul contractului e de nedistins de un notificator obișnuit.

```
public class NotificatorCuIstoric : INotificator
{
    private readonly INotificator interior;
    private readonly string[] istoric = new string[10];
    private int numarMesaje;

    public string Canal => interior.Canal;

    public NotificatorCuIstoric(INotificator interior)
    {
        ArgumentNullException.ThrowIfNull(interior);
        this.interior = interior;
    }

    public void Trimite(string destinatar, string mesaj)
    {
        interior.Trimite(destinatar, mesaj);
        istoric[numarMesaje % istoric.Length] = mesaj;
        numarMesaje++;
    }

    public void AfiseazaIstoric()
    {
        for (int i = 0; i < istoric.Length; i++)
        {
            if (istoric[i] != null)
            {
                Console.WriteLine(istoric[i]);
            }
        }
    }
}
```

Folosire în `Testare` — zero modificări în `MagazinOnline` :

```
NotificatorCuIstoric emailCuIstoric = new(new EmailNotificator());

INotificator[] canale =
[
    emailCuIstoric,
    new SmsNotificator(),
    new ImprimantaBonuri(11, 2022)
];

var magazin = new MagazinOnline(canale);
magazin.AnuntaExpediere("Ion Popescu", ["ion.popescu@example.com", "0712345678", ""], "CMD-10");
emailCuIstoric.AfiseazaIstoric();
```

Ordinea din Trimite contează: întâi delegarea, apoi înregistrarea. Dacă `interior.Trimite` aruncă (email invalid), mesajul eșuat NU intră în istoric — istoricul conține doar ce s-a trimis cu adevărat.

● M1 — Bec: validezi tot, apoi muți starea

ex1/Models/Bec.cs

```
public void SeteazaIntensitate(int procent)
{
    Intensitate = procent;
    EstePornit = true;
}
```

Doar ordinea liniilor s-a schimbat. Setter-ul lui `Intensitate` validează: dacă aruncă, execuția nu mai ajunge la `EstePornit = true` și becul rămâne exact cum era. O operație eșuată nu lasă urme.

● M2 — IPornibil: doar get în contract

ex1/Models/IPornibil.cs + implementările

```
public interface IPornibil
{
    bool EstePornit { get; }

    void Porneste();

    void Opreste();
}
```

```
public class Bec : IPornibil, IReglabil, IRaportor
{
    public bool EstePornit { get; private set; }
    ...
}
```

Mecanism: interfața e fața publică — spune ce poți *citi*. Cum se schimbă starea e treaba exclusivă a lui `Porneste()` / `Opreste()`. Cu `private set` în clasă, nimeni din afară nu mai poate face `bec.EstePornit = true` pe scurtătură. Aceeași schimbare în `Priza` și `Boxa`.

● M3 — Ramburseaza: aceeași condiție ca Plateste

ex2/Models/CardBancar.cs, Numerar.cs

```

public void Ramburseaza(double pret)
{
    if (pret <= 0)
    {
        throw new ArgumentException("Amount must be positive");
    }
    ...
}

```

„Positive” înseamnă strict mai mare ca zero — `<= 0`, ca în `Plateste`. Aceeași regulă de business, aceeași condiție, peste tot.

M4 — ex2: scenariul cu ambele rambursări

ex2/Testare.cs

```

CasaDeMarcat casa = new();
double[] cos = [120.50, 80, 45.99];

CardBancar card = new("ING", 200);
Numerar numerar = new("Cash", 300);
VoucherCadou voucher = new("Voucher", 110);

casa.ProceseazaCos(cos, card);
casa.ProceseazaCos(cos, numerar);
casa.ProceseazaCos(cos, voucher);

card.Ramburseaza(45.99);
numerar.Ramburseaza(20);

try
{
    numerar.Ramburseaza(245.99);
}
catch (InvalidOperationException ex)
{
    Console.WriteLine(ex.Message);
}

```

Pas	Sertar numerar	Rezultat
start	300	—
plăți 120.50 + 80 + 45.99	53.51	toate reușesc
Ramburseaza(20)	33.51	reușește — cerința bifată
Ramburseaza(245.99)	33.51	aruncă „Not enough cash in the drawer”

Două îmbunătățiri față de original: (1) rambursarea reușită stă ÎNAINTE de `try` — dacă ar fi înăuntru și ar arunca, ai afla abia din output că scenariul „bun” n-a rulat; (2) prindem `InvalidOperationException`, nu `Exception` — prinzi exact ce te aștepti să se întâmple; orice altă excepție înseamnă bug și trebuie să se vadă, nu să fie înghițită.

🟡 M5 — ex3: scenariul care chiar demonstrează comutarea

ex3/Testare.cs

```
centru.DistribuieColet("Str. Lalelelor 3", 1);
centru.DistribuieColet("Str. Rozelor 7", 2.5);
centru.DistribuieColet("Bd. Unirii 12", 2);
centru.DistribuieColet("Str. Mihai Viteazu 4", 15);
centru.DistribuieColet("Calea Turzii 90", 25);
centru.DistribuieColet("Str. Memorandumului 2", 25);
centru.DistribuieColet("Piata Unirii 1", 25);
centru.DistribuieColet("Str. Horea 44", 25);
centru.DistribuieColet("Str. Dorobantilor 8", 25);
```

Colet	Cine livrează	De ce
1 kg	drona1	autonomie 14 → 9
2.5 kg	drona1	autonomie 9 → 4
2 kg	drona2	drona1 are sub 5 km — comutarea pe autonomie, vizibilă în output
15 kg	curier1	dronele nu duc peste 3 kg
25 kg x3	robot	curierii nu duc peste 20 kg; contorul ajunge la 3
25 kg (al 4-lea)	nimeni	robotul refuză o dată (reîncărcare) → „No courier available”
25 kg (al 5-lea)	robot	contorul s-a resetat — robotul e din nou disponibil

Merge doar cu Robot corectat (B1): refuzul vine din `PoateLivra`, deci centrul afișează mesajul și programul continuă.

🟡 M6 — ex5: Afișare(), răspunsul de la cerința 4, sortatorul unic

ex5/Models/*.cs + Program.cs

Fiecare clasă își afișează singură datele — `Testare` nu mai duplică formatul în 6 for-uri:

```
public class Elev : IComparabil<Elev>
{
    ...
    public void Afișare()
    {
        Console.WriteLine(Nume + " " + Media);
    }
}
```

Răspunsul cerut în comentariu la finalul lui `Program.cs` (aici comentariul e chiar livrabilul cerut de enunț):


```

/*
Criteriul sta in Elev pentru ca doar Elev stie ce inseamna ordinea pentru el
(media, descrescator). Daca Sortator ar cunoaste criteriile, ar avea nevoie
de cate un if pe fiecare tip concret si ar trebui modificat la FIECARE tip
nou adaugat – exact ce interzice principiul open/closed. Cu criteriul in
spatele contractului, Sortator e scris o singura data si ramane inchis:
orice tip viitor intra doar implementand IComparabil.
*/

```

Despre „același obiect Sortator”: varianta ta generică `Sortator<T>` where `T : IComparabil<T>` e mai bună decât cea din cerință — amestecul de tipuri devine eroare de compilare, nu crash la rulare. Prețul: `Sortator<Elev>` și `Sortator<Produs>` sunt tipuri diferite, deci nu mai există „un singur obiect sortator” — ai un obiect per tip. E un compromis corect; la predare îl declari explicit, ca evaluatorul să vadă o decizie, nu o scăpare.

M7 — Program.cs: o rulare care arată tot

Program.cs

Împreună cu C7 (logica din `Testare` mutată din constructor într-o metodă `Ruleaza()`):

```

internal class Program
{
    private static void Main()
    {
        Console.WriteLine("===== EX1 – Casa inteligenta =====");
        new Interfaces.ex1.Testare().Ruleaza();

        Console.WriteLine("\n===== EX2 – Metode de plata =====");
        new Interfaces.ex2.Testare().Ruleaza();

        Console.WriteLine("\n===== EX3 – Centru de livrari =====");
        new Interfaces.ex3.Testare().Ruleaza();

        Console.WriteLine("\n===== EX4 – Canale de notificare =====");
        new Interfaces.ex4.Testare().Ruleaza();

        Console.WriteLine("\n===== EX5 – Sortatorul universal =====");
        new Interfaces.ex5.Testare().Ruleaza();
    }
}

```

Demo-ul cu figuri (Pasul 1–4, acum comentat) se decommentează și devine prima secțiune. Regula de igienă: în repo nu rămâne cod comentat — ori rulează, ori se șterge (istoricul e în git).

● Cleanups — fix-urile pe scurt

- **C1** · Curier.cs:18 — condiția E deja bool: `return greutateKg <= Capacitate;` (la fel în Robot și Drona).
- **C2** · ILivrador.cs:5 — membrii de interfață sunt publici implicit; se șterge `public : string Identificare();`
- **C3** · Boxa.cs — se șterge `SeteazaIntensitateMinima()` (nefolosită — ModNoapte folosește deja Minim); Porneste / Opreste trec prin proprietatea Volum, nu prin câmpul volum — o singură poartă de intrare, cea cu validare.
- **C4** · Priza.cs:19 — `return "este pornita: " + EstePornit;` — același format ca restul rapoartelor.
- **C5** · MagazinOnline.cs:15–18 — `ArgumentNullException.ThrowIfNull(destinatari);` — simetric cu validarea canalelor din constructor.
- **C6** · CasaDeMarcat.cs — numele metodei intră în output: `Console.WriteLine(metoda.Nume + " – plata cu succes: " + preturi[i]);` iar `Console.Write("\n")` devine `Console.WriteLine()`.
- **C7** · toate Testare.cs — constructorul construiește, Ruleaza() execută: mutarea întregului corp al constructorului într-o metodă `public void Ruleaza()`. Constructorul care face muncă e greu de testat și rulează la fiecare new, vrei-nu-vrei.
- **C8** · EmailNotificator.cs, SmsNotificator.cs — notificatorul nu are nevoie să țină destinatarul ca stare; validarea devine metodă privată chemată din Trimite:

```
public class EmailNotificator : INotificator
{
    public string Canal => "EMAIL";

    public void Trimite(string destinatar, string mesaj)
    {
        ValideazaDestinatar(destinatar);
        Console.WriteLine($"[EMAIL catre {destinatar}] {mesaj}");
    }

    private static void ValideazaDestinatar(string destinatar)
    {
        if (string.IsNullOrEmpty(destinatar) || !destinatar.Contains('@'))
        {
            throw new ArgumentException("Invalid email address");
        }
    }
}
```

Un obiect fără stare inutilă nu poate ajunge într-o stare greșită.